

<https://x-thread.org/t/2077387678241141070>

How to Build the Loops That Just Replaced Entire Prompt Engineering



Hanako @hanakoxbt

1d • 1 tweets



For two years, getting more out of an AI agent meant writing a better prompt.

That era is quietly ending. The best engineers in the world stopped writing prompts a while ago and started writing loops - systems that prompt the agent, check the result, and keep running until the work is done.

Karpathy's overnight script found 20 optimizations he had missed in two decades of tuning by hand. Boris Cherny, who built Claude Code, has not written a line of code this year. Anthropic engineers now merge close to 8x more code per day than they did in 2024.

None of them typed anything smarter. They stopped typing at all.

Here is what a loop actually is, when you need one, when you do not, and how to build your first one.

700 experiments while the human slept

March 2026. Andrej Karpathy pushes three files to GitHub. Around 630 lines of code.

One file held the model. One scored it. One told the agent what to explore and what to leave alone. The agent could only touch the training file. Nothing else.

The cycle was boring on purpose. Read the code, propose a change, train for five minutes, check if the score improved, keep the change if it did, roll back if it did not, then go again.

He pointed it at a model he had already tuned by hand for two decades. He let it run for two days.

It ran 700 experiments. It found 20 improvements. One of them was a missing scalar multiplier in the attention mechanism, subtle, not the kind of bug any linter would ever catch, but exactly the kind of thing a careful engineer could have found and never did.

Shopify's CEO ran the same trick overnight on an internal model. He woke up to a 19 percent quality gain on a model half the size of the previous one.

The insight is not "the AI is smarter." Humans get tired after experiment twelve. Loops do not get tired at all.

— STEP 01 • THE KARPATY LOOP

700 experiments overnight, while the human *slept*

One 630-line script found 20 improvements a two-decade expert had missed.

```

autoresearch • train.py • overnight run • 06:14:22

$ python autoresearch --config program.md --budget 2d
// agent may only edit train.py, prepare.py stays read-only
agent • reading program.md • loading eval • starting search

exp 001 baseline val_bpb 2.5643
exp 047 attention scaling factor -0.0031 kept
exp 198 rope base frequency -0.0044 kept
exp 341 missing scalar in attn -0.0089 kept
exp 512 grad clip threshold -0.0017 kept
exp 663 optimizer momentum -0.0024 kept
exp 700 final commit done

summary • 700 experiments • 20 improvements retained • net -0.0421
goal condition met • halting
  
```

autoresearch • 630 lines • 66k stars on github

humans get tired at experiment 12

A loop is not a prompt on a schedule

Most people hear "loop" and think cron. That misses the whole thing.

A prompt is one instruction. A loop is a goal the AI keeps working toward until it gets there, with nobody in the chair. It plans, executes, verifies its own result, feeds the result back in, and repeats.

Five stages: discover, plan, execute, verify, iterate. Three of them do the actual work.

The verifier is the heart of the loop. Without a real gate on the output, you do not have a loop, you have the agent grading its own homework forever. A gate is a test that passes or fails, a build that compiles or crashes, a linter that returns zero or non-zero. Not a second agent with an opinion.

State is what makes the loop learn. A file on the side, a Linear board, a project log, somewhere outside the conversation that records what was tried and what failed. Tomorrow's run resumes instead of starting from scratch.

A stop condition is what keeps it sane. Every real loop has two exits: the goal is met, or a hard cap fires. Skip this and you have built a machine that runs all night for nothing.

Miss any of the three and you have not built a loop. You have built an expensive script.

The four conditions before you build one

Loops earn their cost only when four things are true at the same time. Miss one and the setup takes more than it returns.

- The task repeats at least weekly. Less than that and the setup cost never amortizes.
- Something can automatically fail the work. A test, a type check, a linter, a build.
- Your token budget can absorb the waste. Loops re-read context, retry, explore.
- The agent has senior engineer tools. Logs, a reproduction environment, the ability to run the code it writes and see what breaks.

The honest version, the part nobody selling loops will tell you: most people do not need the heavy version yet. If you are on a consumer plan trying to run overnight verification loops on serious work, the token bill arrives before the productivity gain does.

Good first loops are the boring ones. CI triage. Dependency bumps. Lint-and-fix passes. Flaky test reproduction. Issue-to-PR drafts on a codebase with strong tests already in place.

Bad first loops are the interesting ones. Architecture rewrites. Auth code. Payments. Anything where "done" is a judgment call and a human still has to weigh in.

The five blocks that make a loop real

Every real loop is assembled from the same five pieces. Claude Code and Codex ship all of them now.

The first is automation. The heartbeat. Something that fires the loop on a schedule or an event. `/loop` runs on a cadence. `/goal` keeps going until a condition you defined actually holds. Without a heartbeat, the loop is a script you ran once and forgot about.

The second is a skill. Project knowledge saved as a `claude-md` file the agent reads on every run. Without it, the loop re-derives your context from zero each cycle. With it, intent compounds. The loop knows your conventions, your build steps, the thing you never do because of that one incident three months ago.

The third is sub-agents. The maker and the checker cannot be the same model. The one that wrote the code is far too generous grading its own work. The one that wrote the article misses its own weak sections. Writer fast and cheap. Reviewer slow and strict. That separation is most of the quality.

The fourth is connectors. The loop opens the pull request, closes the ticket, pings the channel when CI turns green. This is the difference between an agent that says "here is a suggested fix" and a report waiting for you in the morning saying the PR is already merged.

The fifth is the verifier. The test, type check, or build that fails bad work automatically. Everything else is plumbing. This is the block that decides whether the loop helps you or just spends your money.

Stack these together and you get what serious teams now run at scale. Dozens of loops, each owning one narrow job, running side by side while everyone sleeps. One engineer used a fleet loop like this to rewrite an entire codebase from one language to another in about six days, work that would have taken close to a year by hand.

STEP 04 · ANATOMY OF A LOOP

FIVE BLOCKS · IN ORDER

Five pieces. Miss one and you are *burning tokens*.

Every working loop, whether in Claude Code or Codex, is assembled from the same five blocks.

01

automation

THE HEARTBEAT

Fires the loop on a schedule or event. Without it you have a script you ran once.

/loop 30m
/goal condition

02

skill

PERSISTENT MEMORY

Project knowledge saved once, read every run. Intent compounds instead of resetting.

claude-md
SKILL.md

03

sub-agents

MAKER VS CHECKER

The one that wrote the code is too generous grading it. Split writer from reviewer.

writer: fast
reviewer: strict

04

connectors

REAL TOOLS

Opens the PR, closes the ticket, pings the channel when CI turns green.

github · linear
slack · sentry

05

verifier

THE WHOLE TRICK

Test, type check, or build that fails bad work automatically. Everything else is plumbing.

tests · types
lint · build

fire

→

recall

→

write

→

check

→

act

→

gate

ships in claude code · ships in codex · same five blocks

the verifier is not optional

Where loops fail silently

Loops do not crash. They bill you in silence. Two failure modes worth naming, both of which get worse as the loop gets better, not easier.

The first is the Ralph Wiggum loop. Engineer Geoffrey Huntley documented it. The agent decides it is done too early, emits the completion signal on a half-finished job, and the loop exits satisfied. Without a hard objective gate, the loop keeps running the next night, keeps spending, keeps producing work nobody will accept.

The fix is not a smarter agent. It is a dumber gate. A test that passes or fails. A build that compiles or does not. Something with no opinion.

The second is subtler. Comprehension debt. The faster the loop ships code you did not write, the larger the gap between what your repo contains and what you actually understand. A smooth-running loop charges compound interest on that gap. The day you have to debug a system nobody on the team has read costs more than the tokens ever did.

Cognitive surrender comes with it. When the loop runs itself, it is tempting to stop forming an opinion and accept whatever comes back. Designing the loop is the cure when you do it with judgment. It is the accelerant when you do it to avoid thinking. Same action, opposite outcome.

Two people can build the exact same loop and end up in opposite places. One uses it to move faster on work they understand deeply. The other uses it to avoid understanding the work at all. The loop does not know the difference. You do.

— STEP 05 · WHERE LOOPS FAIL SILENTLY // DO NOT SKIP

Loops do not crash. They *bill you in silence.*

Two failure modes get worse as the loop gets better, not easier. If you cannot name them, you have not run one for real yet.

RALPH WIGGUM LOOP

The false finish

agent calls it done on a half-done job

```
> iter 03 · tests failing: 7
> iter 04 · tests failing: 4
> iter 05 · "good enough" // no gate
loop · exits satisfied
reality · 2 tests still red
next night · retries same
token bill · running for nothing
```

FIX objective gate. tests pass, or the loop keeps running. no soft passes.

COMPREHENSION DEBT

The trap that grows

the smoother the loop, the larger the gap

```
> week 01 · loop ships 12 PRs · read 12
> week 02 · loop ships 34 PRs · skim 8
> week 03 · loop ships 67 PRs · skim 3
> week 04 · loop ships 89 PRs · skim 0
gate · catches 94%
the 6% · will need debugging
that day · costs more than tokens
```

FIX read the diffs. spot-check the gate. block the loop from judgment work.

same loop · two people · opposite outcomes

the loop does not know. you do.

Start with one loop, not ten

The mistake everyone makes is trying to build the whole system on day one. Ten loops, a dashboard, a fleet. It collapses by the weekend because you cannot tell which loop did what.

Start with one. Pick the most annoying recurring task you have, the thing you check every morning out of habit, and turn that single job into a loop. Let it run for a few days. Watch where it overreaches. Watch where it misses. Tighten. Then add the second one.

The order matters more than the tools. Get one manual run reliable first, not fast, reliable. Turn that run into a claude-md skill. Wrap the skill in a loop with an objective gate and a hard stop condition. And then, only then, put it on a schedule. Skipping ahead, scheduling something you have not made reliable by hand, is exactly how loops blow up while you sleep.

The metric that decides if the loop is working is not tokens spent or tasks attempted or PRs opened. It is cost per accepted change. If your accepted-change rate is below 50 percent, you are doing the review work the loop was meant to remove, and the loop is losing.

Karpathy stopped writing training code. Cherny stopped prompting. Neither of them stopped thinking. If you take one thing from this, take that. The loop is a system that does the boring 95 percent while you keep your full attention on the 5 percent that actually carries risk.

You are paying for a fleet of agents and using one chat window.

Follow me and subscribe to my Telegram channel: <https://t.me/+75nMf005jRpjMDU1>



Hanako @hanakoxbt

1d • 1 tweets

Building my name | ai researcher | dm is open

Follow @hanakoxbt

SOURCE URL

<https://x-thread.org/t/2077387678241141070>

← Unroll another thread